

Label Taxonomies

GitLab uses a set of standardized labels to allow metrics and logs to be aggregated in a standard way. Matching labels over different exporters and metrics allows the simplification of visualization, analysis and alerting logic across GitLab observability data.

However, for a variety of reasons, the taxonomies of labels used in different GitLab instances differs. For instance, GET instances do not need an **environment**, **shard** or **stage** label, while these are important for GitLab.com.

The **label-taxonomy** library allows different label to be used in different GET environments, while also allowing certain labels to be omitted from environments.

Taxonomy Classification

The following label types of classified in the label taxonomy.

Category	Description	GitLab.com Example
environmentThanos	The environment selector label used for Thanos	<code>env="gprd"</code>
environment	The environment selector used for Prometheus	<code>environment="gprd"</code>
tier	The service tier (sv , db stor etc)	<code>tier="stor"</code>
service	The service identifier (mandatory)	<code>type="gitaly"</code>
stage	Used for identifying staggered deployments within a single environment (main , cny etc)	<code>stage="cny"</code>
shard	An service bulkhead, used to isolating subcomponents of a service against saturation in other subcomponents	<code>shard="maquee"</code>
node	The label used to identify a VM instance	<code>fqdn="file-01..."</code>

Configuring the Label Taxonomy

The label taxonomy should be defined in the `gitlab-metrics-config.libsonnet` configuration file.

This maps a label key to an actual string label value, or **null** if the label is not used in a specific environment.

```
labelTaxonomy:: labelSet.makeLabelSet({
  environmentThanos: null, // No thanos
  environment: null, // Only one environment
  tier: null, // No tiers
  service: 'type',
  stage: null, // No stages
  shard: null, // No shards
})
```

```

        node: 'node',
      }},
    }

```

(Note: for all results in examples are based on the configuration posted above).

Using the API

Labelsets

Label sets are internally defined as a using a bitset. The set of all labels is defined in `labelTaxonomy.labels`.

Labelsets can be created using the `|` operator, for example:

```
local requiredLabelSet = labelTaxonomy.labels.environmentThanos | labelTaxonomy.labels.environment
```

Labelsets can be joined together using the `|` operator.

```
// Union of all labels in both sets
local labelSet3 = labelSet1 | labelSet2;
```

Other operations (subtraction, intersection etc) can similarly be done using bitwise arithmetic.

```
labelTaxonomy.labelTaxonomy(labelSet) --> [labels]
```

The `labelTaxonomy(labelSet)` method will return the actual labels for a given labelSet. Any labels not defined for a given environment will be omitted.

```
local labels = labelTaxonomy.labels;
labelTaxonomy.labelTaxonomy(l.environmentThanos | l.environment | l.service | l.node) => ['t
```

```
labelTaxonomy.labelTaxonomySerialized(labelSet) --> "labels"
```

Similar to `labelTaxonomy(labelSet)`, except this will return a string, suitable for use in PromQL.

```
local labels = labelTaxonomy.labels;
labelTaxonomy.labelTaxonomy(l.environmentThanos | l.environment | l.service | l.node) => 'ty
```

```
getLabelFor(label, default='')
```

Returns the actual label string value for a given label.

```
labelTaxonomy.getLabelFor(labelTaxonomy.labels.stage, default="") ==> ''
```